Microprocessor & Assembly Language

CSE 0611-3221

Flow Control Instructions

Presented By

Saifur Rahman

Lecturer

Dept. of CSE

NBIU

References

Assembly Language Programming and Organization of the IBM PC

By Ytha Yu and Charles Maru

Flow Control Instructions

JUMP and LOOP Instructions: Move control to another part of the program.

Types of Transfer:

- **Unconditional:** Happens without any conditions.
- **Conditional:** Depends on specific status flag settings.

Conditional Jumps

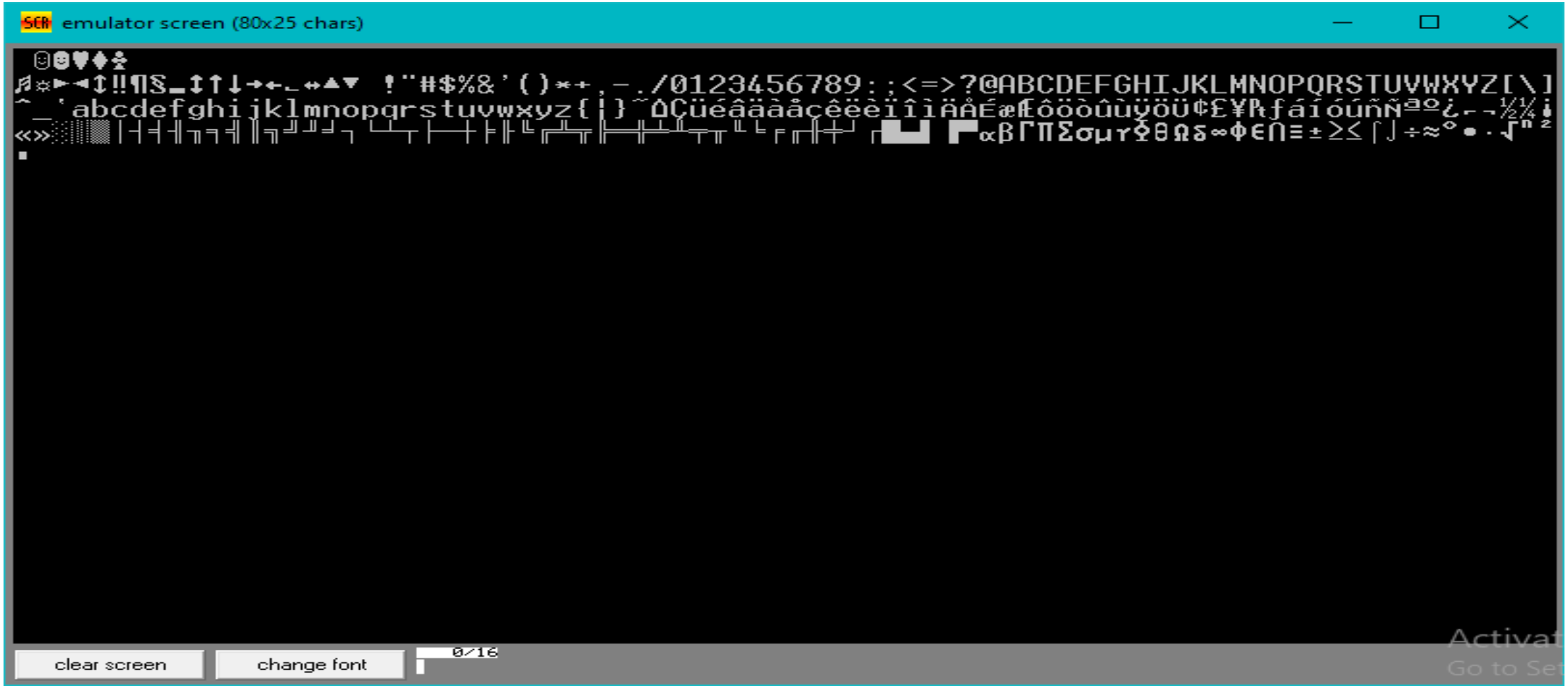
Syntax: `JXXX destination_label`

- For a conditional jump, the CPU checks the **FLAGS register**, which shows the result of the last operation.
- If the jump condition (based on flag settings) is **true**, the CPU changes the **Instruction Pointer (IP)** to the **destination label**, executing the instruction there next.
- If the condition is **false**, the IP stays the same, and the CPU moves to the **next instruction** in sequence.

Conditional Jumps (cont'd)

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
   MOV AH, 2       ; Set AH to 2 (display character function)
   MOV CX, 256     ; Set CX as the counter to 256 characters
   MOV DL, 0       ; Set DL to the ASCII code of the null character (0)
PRINT_LOOP:
   INT 21h         ; Call DOS interrupt to display the character in DL
   INC DL          ; Increment DL to move to the next ASCII code
   DEC CX          ; Decrement counter in CX
   JNZ PRINT_LOOP ; Jump to PRINT_LOOP if CX is not zero
   MOV AH, 4Ch     ; DOS terminate program function
   INT 21h         ; Call DOS interrupt to exit
   MAIN ENDP
END MAIN
```

Conditional Jumps (cont’d)



Conditional Jumps (cont'd)

In the program, the CPU uses the **JNZ PRINT_LOOP** instruction to decide whether to loop back.

The **Zero Flag (ZF)** is checked here:

- If **ZF = 0**, it means CX is not zero, so the program jumps back to **PRINT_LOOP**.
- If **ZF = 1**, it means CX has reached zero, so the loop ends, and the program continues to **MOV AH, 4Ch** to terminate.

Conditional Jumps (cont'd)

There are three categories of jumps:

- **Signed Jumps:** Used when interpreting results as signed values.
- **Unsigned Jumps:** Used when interpreting results as unsigned values.
- **Single-Flag Jumps:** Depend on the settings of individual flags (like Zero or Carry Flag).

Note: Jump instructions check flags but do not modify them.

Conditional Jumps (cont'd)

Table 1: Signed Jumps

Symbol	Description	Conditions for Jump
JG JNLE	Jump if greater than Jump if not less than or equal	ZF = 0 and SF = OF
JGE JNL	Jump if greater than or equal Jump if not less than	SF = OF
JL JNGE	Jump if less than Jump if not greater than or equal	SF <> OF
JLE JNG	Jump if less than or equal Jump if not greater than	ZF = 1 or SF <> OF

Conditional Jumps (cont'd)

Table 2: Unsigned Jumps

Symbol	Description	Conditions for Jump
JA JNBE	Jump if above Jump if below or equal	CF = 0 and ZF = 0
JAE JNB	Jump if above or equal Jump if not below	CF = 0
JB JNAE	Jump if below Jump if not above or equal	CF = 1
JBE JNA	Jump if below or equal Jump if not above	CF = 1 or ZF = 1

Conditional Jumps (cont'd)

Table 3: Single-Flag Jumps

Symbol	Description	Conditions for Jump
JE JZ	Jump if equal Jump if equal to zero	ZF = 1
JNE JNZ	Jump if not equal Jump if not equal to zero	ZF = 0
JC	Jump if carry	CF = 1
JNC	Jump if no carry	CF = 0
JO	Jump if overflow	OF = 1
JNO	Jump if no overflow	OF = 0
JS	Jump if sign negative	SF = 1
JNS	Jump if sign nonnegative	SF = 0
JP/JPE	Jump if parity even	PF = 1
JNP/JPO	Jump if parity odd	PF = 0

The CMP Instruction

Syntax: `CMP destination, source`

- The CMP instruction performs the subtraction of the source operand from the destination operand.
- Both operands cannot be memory locations; one must be a register or immediate value.
- The destination operand cannot be a constant.
- CMP works similarly to the SUB instruction, but unlike SUB, the destination operand remains unchanged.

The CMP Instruction (cont'd)

```
CMP AX, BX
JG BELOW
```

- Where, AX = 7FFFh, and BX = 0001h.
- CMP = 7FFFh – 0001h = 7FFEh = 0111 1111 1111 1110
- Since, ZF = SF = OF = 0. The jump condition is satisfied, so control transfers to label BELOW

The CMP Instruction (cont'd)

- Suppose AX and BX contain signed numbers. Write code to store the largest value in the register CX.

```
MOV    CX, AX      ; put AX into CX
CMP    CX, BX      ; is BX bigger?
JLE    NEXT        ; No, go on
MOV    CX, BX      ; yes, put BX in CX
NEXT:
```

Signed vs Unsigned Jumps

```
CMP AX, BX
JA BELOW
```

Where, AX = 7FFFh, and BX = 8000h.

- 7FFFh > 8000h (signed interpretation)
- 7FFFh < 8000h (unsigned interpretation)
- **JA** (Jump if Above) is used, which performs an unsigned comparison.
- So, The program does not jump to **BELOW** because the unsigned condition is not satisfied.

Working with Characters

Standard ASCII Character Set (0 – 127):

- Either signed or unsigned jumps may be used.
- The sign bit of a byte containing a character code is always 0.

Comparison of Extended ASCII (128-255):

- Use unsigned jumps for comparing extended ASCII characters (codes 80h to FFh).

The JMP Instruction

- Unconditional control transfer.
- Used for loop implementation.
- Syntax: `JMP destination`

Branching Structure

- If-Then
- If-Then-Else
- Case

Fig 1: If-Then-Else

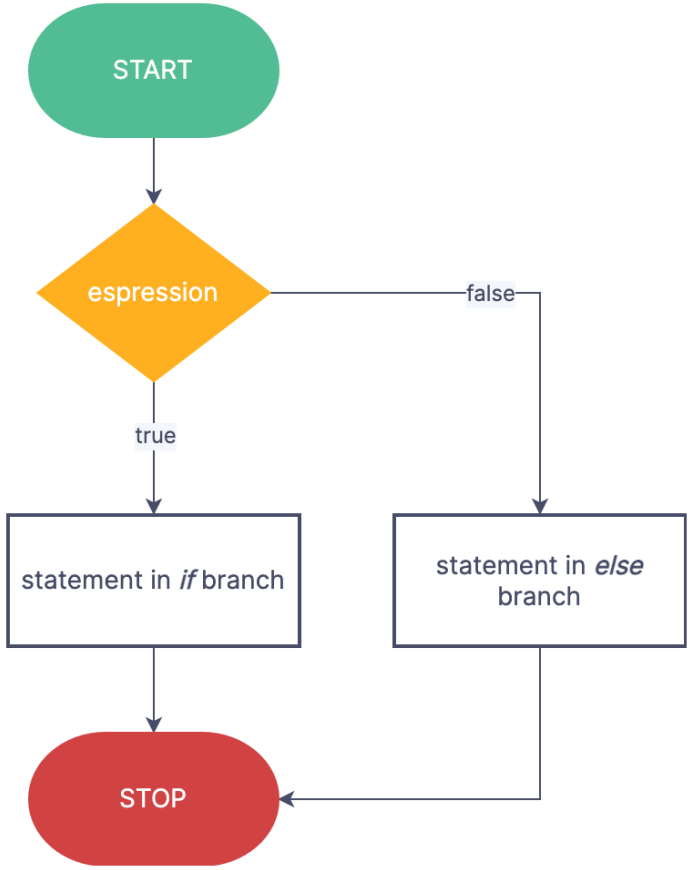
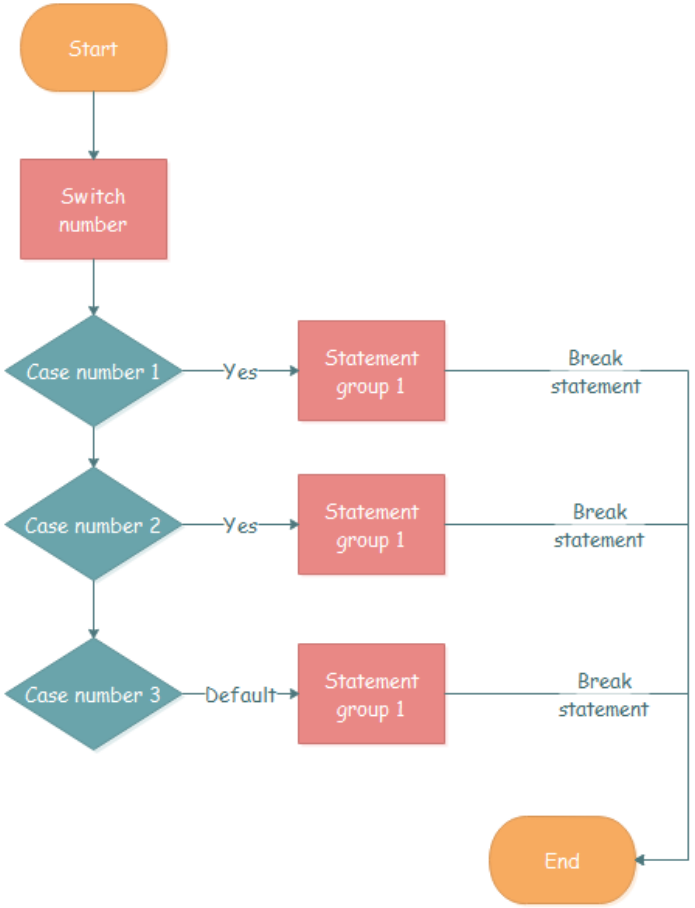


Fig 2: Case



Branching Structure (cont'd)

- Replace the number in AX by its absolute value.

Pseudocode Algorithm:

```
IF AX < 0
 THEN
    replace AX by –AX
END_IF
```

Assembly Code:

```
; if AX < 0
    CMP AX, 0   ; AX < 0?
    JNL END_IF ; No, exit
; then
    NEG AX       ; yes, change sign
END_IF:
```

Branching Structure (cont'd)

- Suppose AL and BL contain extended ASCII characters. Display the one that comes first in the character sequence.

Pseudocode Algorithm:

```
IF AL < BL
 THEN
    Display the character in AL
 ELSE
    Display the character in BL
END_IF
```

Assembly Code:

```
MOV AH, 2            ; prepare to display
; if AL <= BL
     CMP   AL, BL    ; AL <= BL?
     JNBE ELSE_      ; No, display char in BL
; then                ; AL <= BL
     MOV DL, AL      ; move char to display
     JMP DISPLAY     ; go to display
ELSE_:               ; BL < AL
      MOV DL, BL
Display:
      INT 21H
```

Branching Structure (cont'd)

- If AX contains a negative number, put -1 In BX; if AX contains 0, put 0 In BX; if AX contains a positive number, put 1 In BX.

Pseudocode Algorithm:

```
CASE AX
     < 0  : put -1 in BX
     = 0  : put 0 in BX
     > 0  : put 1 in BX
END_CASE
```

Assembly Code:

```
; case AX
     CMP AX, 0    ; test AX
     JL NEGATIVE  ; AX < 0
     JE ZERO      ; AX = 0
     JG POSITIVE  ; AX > 0

NEGATIVE:
     MOV BX, -1 ; -1 in BX
     JMP END_CASE

ZERO:
     MOV BX, 0 ; 0 in BX
     JMP END_CASE

POSITIVE:
     MOV BX, 1 ; 1 in BX

END_CASE:
```

Branching Structure (cont'd)

- If AL contains 1 or 3, display "o"; if AL contains 2 or 4, display "e".

Pseudocode Algorithm:

```
CASE AL
     1, 3 : display 'o'
     2, 4 : display 'e'
END_CASE
```

Assembly Code:

```
; case AL
; 1, 3:
     CMP AL, 1 ; AL = 1?
     JE ODD    ; display 'o'
     CMP AL, 3 ; AL = 3?
     JE ODD    ; display 'o'
; 2, 4:
     CMP AL, 2 ; AL = 2?
     JE EVEN   ; display 'e'
     CMP AL, 4 ; AL = 4?
     JE EVEN   ; display 'e'
ODD:
     MOV DL, 'o'
     JMP Display
EVEN:
     MOV DL, 'e'
Display:
     MOV AH, 2
     INT 21H
```

Branching Structure (AND Condition)

- Read a character, and if it's an uppercase letter, display it.

Pseudocode Algorithm:

```
Read a character (into AL)
IF ('A' <= character) and (character <= 'Z')
THEN
display character
END IF
```

Assembly Code:

```
; Read a character
mov ah, 1
int 21h        ; Character in AL

; if 'A' <= char <= 'Z'
cmp al, 'A'        ; Compare char with 'A'
jl END_IF          ; If char < 'A', skip display
cmp al, 'Z'        ; Compare char with 'Z'
jg END_IF          ; If char > 'Z', skip display

; Then display character
mov ah, 2
mov dl, al
int 21h            ; Display character in AL

END_IF:
```

Branching Structure (OR Condition)

- Read a character. If it's 'a' or 'A', display it; otherwise, terminate the program.

Pseudocode Algorithm:

```
Read a character (into AL)
IF (character = 'a') OR (character = 'A')
THEN
display character
END IF
```

Assembly Code:

```
; Read a character
MOV AH, 1          ; Prepare to read
INT 21H            ; Character in AL


; if (character = 'a') or (character = 'A')
CMP AL, 'a'        ; Is character 'a'?
JE THEN            ; Yes, go to display it
CMP AL, 'A'        ; Is character 'A'?
JE THEN            ; Yes, go to display it
JMP ELSE           ; No, terminate
THEN:
    MOV AH, 2      ; Prepare to display
    MOV DL, AL     ; Load character into DL
    INT 21H        ; Display character
    JMP END_IF     ; Exit
ELSE:
    MOV AH, 4CH    ; DOS exit
    INT 21H        ; Terminate program
END_IF:
```

Looping Structure

- For Loop
- While Loop
- Repeat Loop (do-while)

Fig 3: For Loop

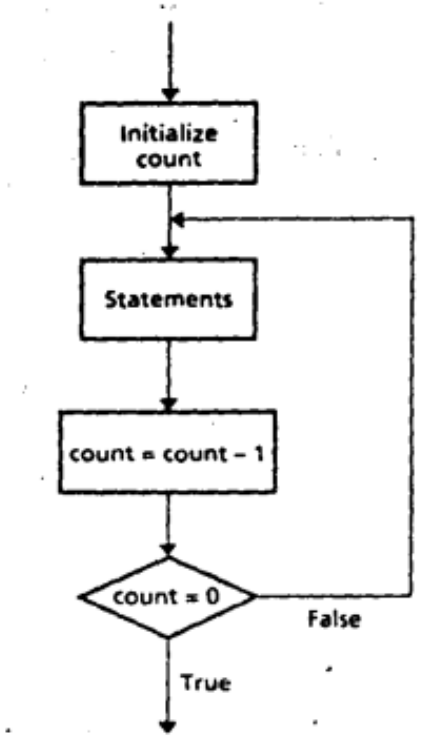


Fig 4: While Loop

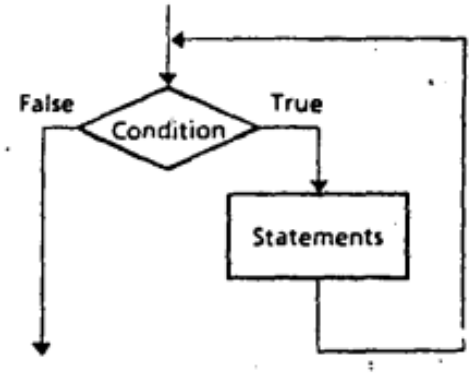
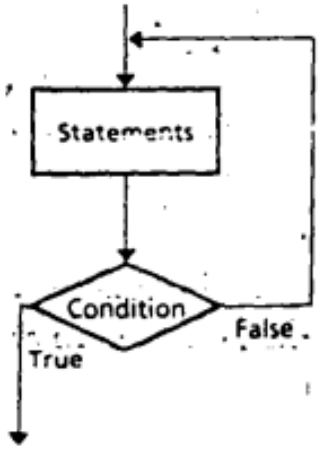


Fig 5: Do-while Loop



Looping Structure (FOR Loop)

- Write a count-controlled loop to display a row of 80 stars.

Pseudocode Algorithm:

```
FOR 80 times DO
     display '*'
END_FOR
```

```
TOP:
    INT 21H
    LOOP TOP   ; Repeat 80 times
```

Assembly Code:

```
; Read a character
MOV CX, 80   ; Loop counter
MOV AH, 2    ; Display character function
MOV DL, '*'  ; Display '*'
```

Looping Structure (While Loop)

- Write some code to count the number of characters in an input line.

Pseudocode Algorithm:

```
Initialize count to 0
Read a character
While character <> carriage_return Do
      count = count + 1
      read a character
END_While
```

Assembly Code:

```
MOV DX, 0          ; DX counts characters
MOV AH, 1          ; Prepare to read a character
INT 21H            ; Character read into AL
```

```
WHILE:
    CMP AL, 0DH   ; if character is CR (0DH)
    JE END_WHILE  ; If yes, exit loop
    INC DX        ; If not CR, increment count
    INT 21H       ; Read next character
    JMP WHILE     ; Loop back

END_WHILE:
```

Looping Structure (Do-while Loop)

- Write some code to read characters until a blank is read.

Pseudocode Algorithm:

```
Repeat
 Read a character
Until character is blank
```

Assembly Code:

```
MOV AH, 1   ; Prepare to read a character
REPEAT:
    INT 21H      ; Character read into AL
    CMP AL, ‘ ‘  ; a blank?
    JNE REPEAT
```

While vs Repeat

WHILE Loop

- Bypasses loop if the condition is initially false.
- Requires two jumps (conditional at the top (break statement), JMP at the bottom).

REPEAT Loop

- Executes the loop body at least once.
- Typically shorter with only one conditional jump at the end.

High-Level Structure

- Prompt the user to enter a line of text. On the next line, display the capital letter entered that comes first alphabetically and the one that comes last. If no capital letters are entered, display "No capital letters". The execution should look like this:

```
Type a line of text:
THE QUICK BROWN FOX JUMPED.
First capital = B Last capital = X
```